TECHNICAL & USER DOCUMENTATION

HearthHOA

A web portal for homeowners associations: announcements, dues, documents, maintenance requests, directories, and AI-assisted drafting, translation and document Q&A.

Version 0.1.0 (Phase 1 MVP)
Generated October 23, 2026 from the `hearthoa` repository (commit `cce5807`)

Contents

1. Overview
2. Technology Stack
3. Getting Started (Local Setup)
4. Configuration & Environment Variables
5. Project Structure
6. Feature Modules
7. AI Features
8. Roles, Permissions & Access Control
9. Data Model
10. API Reference
11. Scheduled Jobs
12. Stubbed Integrations
13. Testing
14. Production Deployment Checklist
15. Troubleshooting

1. Overview

HearthHOA centralizes the day-to-day operations of a homeowners association (HOA) in a single web application. It is built to reduce administrative load on volunteer board members and give residents self-service access to information.

Goals

- Centralize HOA operations in one accessible platform.
- Reduce administrative workload for volunteer board members.
- Improve the resident experience through self-service tools.
- Lay a scalable foundation for future phases (multi-tenant SaaS, online payments).

Phase 1 scope at a glance

Accounts & Auth

Public registration, admin approval queue, bulk resident import, password reset.

Announcements

Critical/General priority, pinning, per-resident email opt-out for general posts.

Dues Tracking

Dues per unit and period, payments, CSV import and period-filtered export.

Document Library

Categorized documents, bulk export, PII review gate before AI indexing.

Maintenance Requests

Residents submit requests; admins move them through a status workflow.

Directories

Board directory, plus an optional resident directory with per-HOA visibility and opt-out.

User Management

Admin CRUD over users, role and status edits, soft delete.

AI Assistance

Announcement drafting and translation; CC&R / document Q&A chatbot (RAG).

Foundational design decisions

- **Tenant-ready.** Phase 1 is single-tenant, but `hoa_id` is on every domain table, so moving to multi-tenant SaaS is a migration rather than a rewrite.
- **Money is split.** `dues` record what is owed; `payments` and `due_payments` record what was received, so Stripe can be added cleanly later.
- **Temporal residency.** `unit_residents` keeps history so past dues stay attributed to the right person.
- **Per-user notification preferences.** Critical announcements bypass opt-outs by design.
- **PII gate.** No document is embedded for the chatbot until its PII review status is cleared.
- **Bounded AI cost.** Per-resident daily message limits and a per-HOA monthly budget cap.

2. Technology Stack

Layer	Technology
Framework	Next.js 15 (App Router) with React 19
Language	TypeScript (strict mode)
Styling	Tailwind CSS with OKLCH design tokens; Radix UI primitives (shadcn-style components)
Database	PostgreSQL with the pgvector extension
ORM	Prisma 5
Server state	TanStack React Query
Forms & validation	react-hook-form + Zod
Authentication	bcryptjs password hashing; JWT (jose) stored in an httpOnly cookie `hh_token`
AI	Anthropic Claude (drafting, translation, chat); OpenAI `text-embedding-3-small` embeddings (1536 dimensions)
File storage	Local disk (`./uploads`) in development; AWS S3 SDK included for production
Email	Resend (optional); Mailpit for local previews
Testing	Vitest (unit and integration), Playwright (end-to-end)
Hosting target	Vercel (cron schedules defined in `vercel.json`); Dockerfile also provided

Architecture

Browser (React 19 + React Query) → Next.js middleware (JWT, role gating) →
App Router pages / `/api` route handlers → Prisma → PostgreSQL + pgvector

Route handlers also call out to file storage, the email provider and the AI providers. Every handler is wrapped by `route()` in `lib/handler.ts`, which converts errors into consistent JSON responses:

- `ApiError` → its own status code and `{ message }`
- Zod validation errors → `422` with `{ message: "Invalid input.", errors: { field: [..] } }`
- Anything else → `500` with `{ message: "Server error." }`

List endpoints accept `pageNumber`, `pageSize` (max 100, default 10) and `search` query parameters, and return metadata with `page`, `pageSize`, `totalCount` and `hasNext`.

3. Getting Started (Local Setup)

Prerequisites

- Node.js (LTS) and npm
- Docker Desktop (for PostgreSQL + pgvector and Mailpit)

Steps

```
# 1. Install dependencies
npm install

# 2. Start Postgres (pgvector) and Mailpit
docker compose up -d

# 3. Create your environment file
cp .env.example .env
#    Optionally add ANTHROPIC_API_KEY, OPENAI_API_KEY, RESEND_API_KEY

# 4. Generate Prisma client, apply migrations, seed the pilot HOA
npx prisma generate
npm run db:setup

# 5. Run the development server
npm run dev
```

Local services and ports

Service	URL / Port
Web application	`http://localhost:3000`
PostgreSQL (pgvector)	`localhost:5432` (user `postgres`, password `postgres`, database `hearthhoa`)
Mailpit web UI	`http://localhost:8025`
Mailpit SMTP	`localhost:1025`

Seeded demo accounts

The seed creates the pilot HOA `sunset-ridge` with two accounts:

Role	Email	Password
Admin	`admin@sunset-ridge.test`	`changeme1234`
Resident	`resident@sunset-ridge.test`	`changeme1234`

Port 5432 already in use? If another PostgreSQL instance is running, map the container to a different host port (for example `"5433:5432"` in `docker-compose.yml`) and update `DATABASE_URL` in `.env` to match.

npm scripts

Command	Purpose
`npm run dev`	Start the development server
`npm run build`	Production build
`npm run start`	Serve the production build
`npm run lint`	ESLint with zero warnings allowed
`npm run typecheck`	TypeScript check without emitting
`npm run db:setup`	Run migrations and seed
`npm run prisma:generate`	Regenerate the Prisma client
`npm run prisma:migrate`	Create/apply a development migration
`npm run prisma:seed`	Seed only
`npm run test`	Unit tests (Vitest)
`npm run test:integration`	Integration tests against a database
`npm run test:e2e`	Playwright end-to-end tests

4. Configuration & Environment Variables

Variable	Required	Description
`DATABASE_URL`	Yes	PostgreSQL connection string. The database must have the pgvector extension.
`JWT_SECRET`	Yes	Secret used to sign session tokens. Use a 48-byte random value in production (`openssl rand -base64 48`).
`NEXT_PUBLIC_API_URL`	Yes	Base URL for API calls from the browser; `/api` for same-origin.
`APP_URL`	Recommended	Public URL of the app, used when building links in emails.
`CRON_SECRET`	Production	Bearer token required by `/api/cron/*`. If unset, cron routes are open in development and refused in production.
`ANTHROPIC_API_KEY`	Optional	Enables real AI drafting, translation and chat. Stubbed when absent.
`OPENAI_API_KEY`	Optional	Enables real embeddings. Zero vectors are returned when absent.
`RESEND_API_KEY`	Optional	Enables real email sending. Emails are logged to the console when absent.

Per-HOA settings (database)

Field on `hoas`	Purpose
`ai_monthly_budget_cents`	Monthly AI spend cap. The system alerts at 80% and gates at 100%.
`ai_per_user_daily_messages`	Chatbot messages a resident may send per day (default 20).
Directory visibility	`off`, `admins_only` or `all_residents`.
Dues cadence	`monthly`, `quarterly` or `annual`.

5. Project Structure

```
app/
  page.tsx                      Landing / login page
  register/                     Public registration
  reset-password/               Password reset
  sandbox/                      Component sandbox (always accessible)
  (app)/                        Authenticated area
    dashboard/
    announcements/  dues/  documents/  maintenance-requests/
    board-members/  residents/  users/  roles-and-permissions/
    translations/   documents/pii-review/
    import/residents/  import/dues/
    settings/notifications/
  api/                          Route handlers (see Section 10)
components/
  ds/  ui/                      Design-system and UI primitives
  Sidebar.tsx  ChatbotWidget.tsx
hooks/                          React hooks
lib/
  api.ts          Client-side API wrapper (axios)
  auth.ts  session.ts  session-types.ts   Auth and session helpers
  handler.ts      route() wrapper, body parsing, pagination
  permissions.ts  Role -> permission map (source of truth)
  prisma.ts       Prisma client singleton
  storage.ts      File storage abstraction
  parsers.ts      CSV/text parsers for imports
  chunking.ts     Document chunker for RAG
  pii-scan.ts     PII detection for documents
  embeddings.ts   Embedding provider wrapper
  email.ts  email-templates.ts            Email delivery and templates
  format.ts  utils.ts
middleware.ts     Authentication and admin-route gating
prisma/
  schema.prisma  migrations/  seed.ts
tests/
  *.test.ts  integration/  e2e/
```

6. Feature Modules

6.1 Accounts & Authentication

- **Login** on the landing page (`/`). A successful login sets the httpOnly `hh_token` JWT cookie.
- **Registration** at `/register`. New users start with status `pending` and must be approved by an admin.
- **Approval queue** at `/users?status=pending`, using the approve/reject endpoints.
- **Password reset** via `/api/auth/forgot-password` (emails a link) and `/reset-password`.
- **Bulk resident import** at `/import/residents` for onboarding from a spreadsheet.
- User statuses: `pending`, `active`, `disabled`.

6.2 Dashboard

The landing page after login, at `/dashboard`. Residents who try to open an admin-only page are redirected here with an "admin-only" notice.

6.3 Announcements

- Priority is `general` or `critical`. Critical announcements reach every resident, regardless of email opt-outs.
- Announcements can be pinned until a date; an hourly job unpins expired ones.
- Admins can draft with AI and generate translations (see Section 7).
- Pages: list, view, create, and edit (create and edit are admin-only).

6.4 Dues Tracking

- Dues statuses: `unpaid`, `partial`, `paid`, `late`, `waived`.
- Payment methods: `cash`, `check`, `transfer`, `card`, `ach`, `waiver`.
- Payments link to dues through `due_payments`, so one payment can cover several dues.
- CSV import from legacy spreadsheets (`/import/dues`) and a period-filtered CSV export.
- Residents see only their own dues; admins see all of them.
- A daily job marks past-due `unpaid` / `partial` dues as `late`.

6.5 Document Library

- Categories: `bylaws`, `ccrs`, `minutes`, `financials`, `forms`, `other`.
- Files are uploaded through `/api/uploads` and served through `/api/files/[key]`.
- One-click bulk export for board archiving.
- **PII review:** each document has a status of `pending`, `acknowledged_safe`, `flagged` or `excluded`. Only `acknowledged_safe` documents may be ingested into the chatbot index. Admins review flagged items at `/documents/pii-review`.

6.6 Maintenance Requests

- Categories: `landscaping`, `plumbing`, `electrical`, `roads`, `pool`, `other`.
- Workflow: `new` → `in_progress` → `resolved` → `closed`.
- Residents create and view their own requests; admins see all requests and change their status.
- A daily job closes requests that have been `resolved` for more than 14 days.

6.7 Board & Resident Directories

- **Board directory** (`/board-members`) is readable by everyone; admins create and edit entries.
- **Resident directory** (`/residents`) follows the HOA visibility setting. Residents can opt themselves out via `PATCH /api/me/directory`.

6.8 User Management

Admin-only CRUD over portal users at `/users`: change role and status, soft-delete users, and approve or reject pending registrations.

6.9 Roles & Permissions page

A read-only matrix at `/roles-and-permissions`, backed by `lib/permissions.ts`, that shows which permissions each role has.

6.10 Notification Settings

At `/settings/notifications`, each user sets per-type, per-channel (`email`, `sms`) preferences. Opt-outs take effect immediately; critical announcements are always delivered.

7. AI Features

7.1 Announcement Drafting

`POST /api/ai/draft` uses Claude (`claude-sonnet-4-6`, up to 1,200 output tokens) to draft an announcement from an admin's notes. Without `ANTHROPIC_API_KEY`, it returns a hand-written stub.

7.2 Announcement Translation

- `POST /api/announcements/[id]/translate` produces a translation with Claude (up to 1,500 output tokens).
- New translations are labelled **"machine-translated, unverified"** until a bilingual reviewer signs off.
- Reviewers work through the queue at `/translations` and approve with `POST /api/translations/[id]/review`.

7.3 CC&R / Document Q&A Chatbot

The chatbot widget lets residents ask questions about HOA documents through retrieval-augmented generation (RAG).

Upload document → PII scan → Admin review: acknowledged_safe → Ingest: chunk + embed → pgvector search → Claude answer with citations

- **PII scan** (`lib/pii-scan.ts`) uses patterns to detect email addresses, phone numbers, Social Security numbers and street addresses.
- **Chunking** (`lib/chunking.ts`) splits text into chunks of about 800 tokens with a 100-token overlap.
- **Embeddings** use OpenAI `text-embedding-3-small` (1536 dimensions), stored in `document_chunks`.
- **Rate limit:** each resident may send `ai_per_user_daily_messages` questions per UTC day. Beyond that the API returns `429`.
- **Cost tracking:** token counts and `cost_cents` are recorded on every `chat_messages` row.
- **Safety contract:** when the answer is not in the documents, the bot says so and points the resident to the board.

> **Current status:** the chat endpoint records sessions and messages and enforces rate limits. The retrieval-and-answer step still returns a fallback message until document ingestion is fully connected.

8. Roles, Permissions & Access Control

Every API route checks permission strings derived from the user's role, not the role itself. Adding a role means extending the map in `lib/permissions.ts`; routes do not change.

Roles

Role	Active in Phase 1	Description
Resident	Yes	Homeowners and verified residents. Read-mostly access; can submit their own maintenance requests and use the chatbot.
Admin	Yes	Full board access. Manages every module and every other user.
Super admin	Modelled	HOA owner or platform operator. Will be the only role able to manage custom roles in Phase 2.
Treasurer	Modelled	Resident permissions plus all dues, payments, imports and audit access.
Secretary	Modelled	Resident permissions plus announcements, translations, documents and AI drafting.
Maintenance lead	Modelled	Resident permissions plus viewing all maintenance requests and changing their status.

"Modelled" roles are defined in the permission map. They take effect once they are added to the Prisma `Role` enum, which is a one-line change.

Permission matrix

Permission	Resident	Treasurer	Secretary	Maint. lead	Admin / Super
`announcements:read`	✓	✓	✓	✓	✓
`announcements:write`, `:translate`			✓		✓
`documents:read`	✓	✓	✓	✓	✓
`documents:write`, `:export`			✓		✓
`documents:ingest`, `:pii-review`					✓
`dues:read:own`	✓	✓	✓	✓	✓
`dues:read:all`, `dues:write`, `dues:export`, `payments:write`		✓			✓
`maintenance:read:own`, `:write:own`	✓	✓	✓	✓	✓
`maintenance:read:all`, `:write:state`				✓	✓
`board:read`, `residents:read`, `ai:chat`	✓	✓	✓	✓	✓
`board:write`					✓

`translations:review`, `ai:draft`	✓	✓
`imports:run`, `audit:read`	✓	✓
`users:*`, `roles:*`, `settings:hoa`		✓

Page-level gating (middleware)

- Public pages: `/`, `/login`, `/register`, `/forgot-password`, `/reset-password`. Signed-in users who open these are redirected to `/dashboard`.
- Unauthenticated requests to protected pages are redirected to `/?next=<path>`. An invalid token clears the cookie.
- Pages residents cannot open: `/users`, `/roles-and-permissions`, `/translations`, `/documents/pii-review`, `/import/*`, and any path ending in `/create` or `/edit`.
- API routes are not covered by the middleware; each handler checks the session and permissions itself.

9. Data Model

Defined in `prisma/schema.prisma`. Every domain table carries `hoa_id`.

Model	Purpose
`Hoa`	The association: name, slug, dues cadence, directory visibility, AI budget and rate limits.
`User`	Portal account: email, password hash, role, status, directory opt-out.
`Unit`	A property or unit within the HOA.
`UnitResident`	Temporal link between users and units (residency history).
`Announcement`	Post with priority, pinning and author.
`AnnouncementTranslation`	Translated copy with machine-translated/verified review state.
`Due`	Amount owed by a unit for a period, with status and due date.
`Payment`	Money received, with method.
`DuePayment`	Allocates payments to dues (many-to-many).
`Document`	Uploaded file with category and PII review status.
`DocumentChunk`	Text chunk with a pgvector embedding for RAG.
`MaintenanceRequest`	Resident request with category, status and resolution timestamps.
`BoardMember`	Board directory entry.
`ChatSession` / `ChatMessage`	Chatbot conversations, token usage, cost and feedback.
`UserNotificationPreference`	Per-user, per-type, per-channel opt-in/out.
`NotificationLog`	Record of sent, failed or bounced notifications.
`AuditLog`	Audit trail of sensitive actions.

Enumerations

Enum	Values
Role	resident, admin
UserStatus	pending, active, disabled
DuesCadence	monthly, quarterly, annual
DirectoryVisibility	off, admins_only, all_residents
AnnouncementPriority	general, critical
DuesStatus	unpaid, partial, paid, late, waived
PaymentMethod	cash, check, transfer, card, ach, waiver

DocumentCategory	bylaws, ccrs, minutes, financials, forms, other
PiiReviewStatus	pending, acknowledged_safe, flagged, excluded
MaintenanceCategory	landscaping, plumbing, electrical, roads, pool, other
MaintenanceStatus	new, in_progress, resolved, closed
ChatRole	user, assistant
FeedbackTone	positive, negative
NotificationChannel	email, sms
NotificationStatus	sent, failed, bounced

10. API Reference

All endpoints live under `/api` and use JSON. Authentication uses the `hh_token` cookie set by the login endpoint.

Authentication

Method	Path	Description
POST	`/api/auth/login`	Log in with `{ email, password }`; sets the session cookie.
POST	`/api/auth/logout`	Clear the session.
GET	`/api/auth/me`	Current user.
POST	`/api/auth/register`	Public registration (status `pending`).
POST	`/api/auth/forgot-password`	Send a password reset email.
POST	`/api/auth/reset-password`	Set a new password with a reset token.

Modules (standard CRUD)

Each of these resources exposes `GET` / `POST` on the collection and `GET` / `PATCH` / `DELETE` on `/[id]`:

Resource	Extra endpoints
`/api/announcements`	`POST /[id]/translate`
`/api/dues`	`GET /export`
`/api/documents`	`GET /export`, `POST /[id]/ingest`, `POST /[id]/pii-review/clear`
`/api/maintenance-requests`	-
`/api/board-members`	-
`/api/users`	`POST /[id]/approve`, `POST /[id]/reject`

Other endpoints

Method	Path	Description
POST	`/api/ai/draft`	AI announcement draft.
POST	`/api/ai/chat`	Ask the chatbot `{ question, sessionId? }`. Returns `answer`, `citations` and `remainingToday`.
GET	`/api/translations`	Translation review queue.
POST	`/api/translations/[id]/review`	Mark a translation as reviewed.
POST	`/api/imports/residents`	Bulk import residents from CSV.

POST	`/api/imports/dues`	Bulk import dues from CSV.
POST	`/api/uploads`	Upload a file.
GET	`/api/files/[key]`	Download a stored file.
GET	`/api/residents`	Resident directory (respects visibility settings).
PATCH	`/api/me/directory`	Opt in or out of the resident directory.
GET / PATCH	`/api/notification-preferences`	Read or update the current user's notification preferences.
GET	`/api/roles`	Role and permission matrix.
GET	`/api/units`	List units.
GET	`/api/cron/*`	Scheduled jobs (see Section 11).

Example

```
curl -c cookies.txt -H "Content-Type: application/json" \
  -d '{"email":"admin@sunset-ridge.test","password":"changeme1234"}' \
  http://localhost:3000/api/auth/login

# => {"user":{"id":"...","email":"admin@sunset-ridge.test","name":"Sam Adler","role":"admin"}}

curl -b cookies.txt "http://localhost:3000/api/announcements?pageNumber=1&pageSize=10"
```

11. Scheduled Jobs

Defined in `vercel.json`. Each job requires the header `Authorization: Bearer $CRON_SECRET` in production.

Endpoint	Schedule (UTC)	Action
`/api/cron/auto-unpin`	Hourly (`0 * * * *`)	Unpins announcements whose `pinnedUntil` has passed.
`/api/cron/auto-close-maintenance`	Daily 03:00 (`0 3 * * *`)	Closes requests resolved more than 14 days ago.
`/api/cron/late-dues`	Daily 04:00 (`0 4 * * *`)	Marks `unpaid` / `partial` dues past their due date as `late`.

12. Stubbed Integrations

The app runs fully in development without external credentials; these pieces fall back automatically:

Area	Module	Behaviour without credentials
AI drafting	`POST /api/ai/draft`	Returns a hand-written stub draft.
AI chat	`POST /api/ai/chat`	Records the question; answers "I couldn't find this in our HOA documents, please contact the board."
Embeddings	`lib/embeddings.ts`	Returns 1536-dimension zero vectors. The pipeline runs, but retrieval results are meaningless.
Email	`lib/email.ts`	Logs email payloads to the console.
Storage	`lib/storage.ts`	Writes files to `./uploads/`.

13. Testing

- **Unit tests** (`npm run test`) cover chunking, email templates, embeddings, formatting, the route handler, PII scanning and sessions.
- **Integration tests** (`npm run test:integration`) run API routes against a test database (`tests/integration`).
- **End-to-end tests** (`npm run test:e2e`) use Playwright (`tests/e2e`). Target flows are login, posting an announcement, uploading a document, submitting a maintenance request and a chatbot conversation.

14. Production Deployment Checklist

> Do not ship without working through this list.

Infrastructure

- Provision managed PostgreSQL (Neon, Supabase or RDS) with the pgvector extension enabled.
- Provision object storage (S3, R2 or Supabase Storage) and switch `lib/storage.ts` to signed URLs; put a CDN in front of it.
- Set `JWT_SECRET` (48 random bytes) and `CRON_SECRET`.

Email

- Choose a provider (Resend, Postmark or SendGrid) and connect it in `lib/email.ts`.
- Add SPF, DKIM and DMARC records; send a test of every template.

Vendors & compliance

- Sign Data Processing Addenda with Anthropic, OpenAI and the email provider; document all subprocessors.
- Have an HOA-law attorney review state-specific rules (e.g. Davis-Stirling, Florida Ch. 720, Texas Property Code).
- Confirm CCPA or state-equivalent resident rights (export, deletion).

Security

- Force HTTPS with HSTS; set the `secure` cookie flag in production.
- Rate-limit `/api/auth/login` and `/api/auth/forgot-password` at the edge.
- Configure Sentry; run `npm audit` and resolve high-severity issues.

Observability & accessibility

- Uptime monitoring on `/api/auth/me`; a cost dashboard from `chat_messages.cost_cents`.
- Target WCAG 2.1 AA (axe/Lighthouse), keyboard navigation on every form, and required alt text on images.

AI

- Set `ANTHROPIC_API_KEY` and `OPENAI_API_KEY`. The pgvector column dimension must match the model (1536).
- Set `ai_monthly_budget_cents` and `ai_per_user_daily_messages` for the HOA.
- Ingest the pilot HOA's real CC&Rs and tune the PII thresholds before launch.

Testing & operations

- Run the tests in CI and block merges when they fail.
- Set up an on-call rotation and verify backups by restoring a point-in-time copy.

15. Troubleshooting

Symptom	Fix
`failed to connect to the docker API`	Start Docker Desktop and wait for the engine to be ready, then run `docker compose up -d` again.
`Bind for 0.0.0.0:5432 failed: port is already allocated`	Another PostgreSQL is using 5432. Map the container to another host port (e.g. 5433) and update `DATABASE_URL`.
Prisma client errors after `npm install`	Run `npx prisma generate`. npm may block package install scripts that would otherwise do this.
Redirected to `/dashboard?notice=admin-only`	The page requires the admin role. Log in as an admin.
Chatbot returns `429`	The resident hit the daily message limit; it resets at 00:00 UTC.
Chatbot always says it couldn't find an answer	Set the AI keys and ingest documents that have been marked `acknowledged_safe`.
Cron endpoint returns `403`	Send `Authorization: Bearer $CRON_SECRET`.
No emails received locally	Without `RESEND_API_KEY`, emails are logged to the server console. Check the terminal running `npm run dev`.